

06-MLflow

1 MLflow: ML Lifecycle Platform

Document Version: 1.0
Generated: December 4, 2025
Standard: MLflow Protocol & Format
Status: Industry Standard

1.1 Overview

MLflow is an open-source platform for managing the complete machine learning lifecycle including experimentation, reproducibility, and deployment. It provides standardized APIs and formats for tracking experiments, packaging models, managing registries, and serving predictions.

1.1.1 Key Features

- **Experiment Tracking:** Log metrics, parameters, and artifacts
- **Model Registry:** Centralized model store with versioning
- **MLProject Format:** Reproducible project structure
- **Model Flavors:** Support for multiple frameworks
- **Deployment:** Package and serve models
- **Reproducibility:** Version code, data, and dependencies

1.1.2 Primary Use Cases

1. **Experiment Management:** Track ML experiments
 2. **Model Registry:** Centralize model storage
 3. **Reproducible Research:** Version entire projects
 4. **Model Packaging:** Create deployable packages
 5. **Comparison & Collaboration:** Share results across teams
-

1.2 Authoritative References

- **MLflow Documentation:** <https://mlflow.org/docs/latest/>
 - **GitHub Repository:** <https://github.com/mlflow/mlflow>
 - **MLflow Protocol:** <https://mlflow.org/docs/latest/rest-api.html>
 - **Model Format:** <https://mlflow.org/docs/latest/models.html>
-

1.3 Format Structure

1.3.1 MLflow Run Directory Structure

```

experiment/
├─ meta.yaml          # Experiment metadata
└─ runs/
    └─ {run_id}/
        ├─ meta.yaml    # Run metadata
        ├─ params/      # Parameters
        │   ├─ param1
        │   └─ param2
        ├─ metrics/     # Metrics
        │   ├─ loss
        │   ├─ accuracy
        │   └─ f1_score
        ├─ tags/        # Tags
        └─ artifacts/   # Outputs
            ├─ model/    # Saved model
            └─ plots/    # Visualizations

```

1.3.2 Meta File Format (YAML)

Real example (meta.yaml):

```

artifact_uri: s3://my-bucket/mlflow/experiment/0/run_id/artifacts
end_time: 1733308800000
entry_point_name: null
experiment_id: '0'
lifecycle_stage: active
run_id: abc123def456
run_uuid: abc123def456
source_name: training.py
source_type: LOCAL
source_version: abc123def456
start_time: 1733304200000
status: FINISHED
tags:
  - data_version: v1.0.0
  - user: data_scientist
user_id: jane.doe

```

1.3.3 MLProject Format (YAML)

Pseudo-structure (logical):

```

name: {project_name}
conda_env: conda.yaml
entry_points:
  main:
    command: "python train.py"
    parameters:
      - param1
      - param2
  evaluate:
    command: "python evaluate.py"
    parameters:
      - model_path

```

Real example (MLproject):

```
name: image-classification
conda_env: conda.yaml
entry_points:
  train:
    command: "python train.py
              --epochs {epochs}
              --batch_size {batch_size}
              --learning_rate {learning_rate}"
    parameters:
      epochs:
        type: int
        default: 10
      batch_size:
        type: int
        default: 32
      learning_rate:
        type: float
        default: 0.001

  evaluate:
    command: "python evaluate.py
              --model_uri {model_uri}
              --data_path {data_path}"
    parameters:
      model_uri:
        type: str
      data_path:
        type: str
        default: "data/test"
```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Track Experiment

Workflow:

```

import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Start experiment
mlflow.set_experiment("classification_exp")

with mlflow.start_run():
    # Log parameters
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 10)

    # Train model
    model = RandomForestClassifier(n_estimators=100, max_depth=10)
    model.fit(X_train, y_train)

    # Log metrics
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    mlflow.log_metric("accuracy", accuracy)

    # Log model
    mlflow.sklearn.log_model(model, "model")

    # Log artifact
    import json
    with open("summary.json", "w") as f:
        json.dump({"accuracy": accuracy}, f)
    mlflow.log_artifact("summary.json")

```

1.4.2 Use-Case 2: Model Registry & Deployment

Workflow:

```

import mlflow.pyfunc

# Register model to registry
result = mlflow.register_model(
    model_uri="runs:/abc123/model",
    name="production-classifier"
)

# Transition to staging
client = mlflow.tracking.MlflowClient()
client.transition_model_version_stage(
    name="production-classifier",
    version=1,
    stage="Staging"
)

# Load and predict
model = mlflow.pyfunc.load_model(
    "models:/production-classifier/Staging"
)
predictions = model.predict(X_test)

```

1.4.3 Use-Case 3: Reproducible Project

Structure:

```
project/
├─ MLproject
├─ conda.yaml
├─ train.py
├─ evaluate.py
├─ data/
│   └─ raw.csv
```

Run:

```
# Execute with MLflow
mlflow run . -P epochs=20 -P batch_size=64
```

1.5 Examples

1.5.1 Example 1: Complete Training Script

```

import mlflow
import mlflow.tensorflow
import tensorflow as tf
import numpy as np
from datetime import datetime

mlflow.set_experiment("neural_network_training")

with mlflow.start_run(run_name=f"run_{datetime.now().strftime('%Y%m%d_%H%M%S')}"):

    # Log parameters
    params = {
        "layers": 3,
        "units": 128,
        "dropout": 0.2,
        "learning_rate": 0.001,
        "epochs": 50,
        "batch_size": 32
    }
    mlflow.log_params(params)

    # Build model
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(128, activation='relu', input_shape=(28,)),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train with logging
    history = model.fit(
        X_train, y_train,
        epochs=50,
        batch_size=32,
        validation_split=0.2,
        verbose=0
    )

    # Log metrics
    test_loss, test_accuracy = model.evaluate(X_test, y_test)
    mlflow.log_metric("test_accuracy", test_accuracy)
    mlflow.log_metric("test_loss", test_loss)
    mlflow.log_metric("final_train_loss", history.history['loss'][-1])

    # Log training history
    for epoch in range(len(history.history['loss'])):
        mlflow.log_metric("train_loss", history.history['loss'][epoch], step=epoch)
        mlflow.log_metric("train_accuracy", history.history['accuracy'][epoch], step=epoch)

    # Log model
    mlflow.tensorflow.log_model(model, "model")

    # Log artifacts
    model.save("model_artifact.h5")
    mlflow.log_artifact("model_artifact.h5")

    # Log metadata
    mlflow.set_tag("model_type", "neural_network")
    mlflow.set_tag("dataset", "mnist")
    mlflow.set_tag("status", "production_ready")

print("Experiment complete!")

```

1.5.2 Example 2: MLProject Configuration

```
name: image_segmentation

python_env: python_env.yaml

entry_points:
  train:
    command: "python src/train.py
      --data_dir {data_dir}
      --output_dir {output_dir}
      --epochs {epochs}
      --batch_size {batch_size}
      --learning_rate {learning_rate}
      --augmentation {augmentation}"
    parameters:
      data_dir:
        type: path
        default: "data/raw"
      output_dir:
        type: path
        default: "outputs"
      epochs:
        type: int
        default: 100
      batch_size:
        type: int
        default: 16
      learning_rate:
        type: float
        default: 0.0001
      augmentation:
        type: bool
        default: true

  evaluate:
    command: "python src/evaluate.py
      --model_uri {model_uri}
      --test_dir {test_dir}"
    parameters:
      model_uri:
        type: str
      test_dir:
        type: path
        default: "data/test"

  deploy:
    command: "python src/deploy.py
      --model_uri {model_uri}
      --serve_port {serve_port}"
    parameters:
      model_uri:
        type: str
      serve_port:
        type: int
        default: 8000
```

1.5.3 Example 3: REST API Query

Tracking Server API:

```
# Get experiments
curl http://localhost:5000/api/2.0/mlflow/experiments/list

# Get runs
curl http://localhost:5000/api/2.0/mlflow/experiments/get?experiment_id=0

# Get run details
curl http://localhost:5000/api/2.0/mlflow/runs/get?run_id=abc123

# Log metric
curl -X POST http://localhost:5000/api/2.0/mlflow/runs/log-metric \
  -H "Content-Type: application/json" \
  -d '{
    "run_id": "abc123",
    "key": "accuracy",
    "value": 0.95,
    "timestamp": 1733308800000,
    "step": 10
  }'
```

1.6 Tools & Ecosystem

Tool	Description	Homepage
MLflow Tracking	Experiment tracking	https://mlflow.org/docs/latest/tracking.html
MLflow Registry	Model registry	https://mlflow.org/docs/latest/model-registry.html
MLflow Projects	Project management	https://mlflow.org/docs/latest/projects.html
MLflow Models	Model flavors	https://mlflow.org/docs/latest/models.html
Databricks	Hosted MLflow	https://databricks.com/
Amazon SageMaker	AWS ML service	https://aws.amazon.com/sagemaker/

Navigation: [Back to Index](#) | [Previous: TensorFlow Serving](#) | [Next: Hugging Face Hub](#)